

IUT d'Aix-Marseille – Département Informatique

BUT Informatique – 2^e année

R3.07 – Bases de données

TD3 : contraintes et déclencheurs

Séance de 4 heures – travail sur feuille

Base support : **mini-pagila** (identique aux TD1 et TD2)

Version étudiants

24 septembre 2026

Déroulement de la séance

Les deux premières séances portaient sur ce que l'on *demande* à la base : l'interroger (TD1), puis programmer des traitements en son sein (TD2). Cette troisième séance porte sur ce que la base *garantit* : comment empêcher qu'une donnée incohérente y entre, quelle que soit l'application ou l'utilisateur qui écrit.

Deux mécanismes, à mobiliser dans cet ordre :

Section 1 – les **contraintes natives** (PRIMARY KEY, UNIQUE, NOT NULL, CHECK, FOREIGN KEY et ses modes ON DELETE / ON UPDATE) : déclaratives, vérifiées par le SGBD, peu coûteuses – *toujours à privilégier* ;

Section 2 – les **déclencheurs** (*triggers*) : du code procédural déclenché automatiquement par une modification de données, qui permet d'exprimer ce qu'une contrainte native ne peut pas dire, ou de maintenir automatiquement une donnée dérivée.

La séance se déroule **sur feuille**. Comme aux TD précédents, l'enjeu est de *prédire* le comportement du SGBD, ici pour des instructions qui *modifient* la base : après tel DELETE, quelles lignes restent ? Quelle erreur est levée, et par quelle contrainte ?

Deux ressources à garder sous les yeux : l'**annexe A** (aide-mémoire de la syntaxe) et l'**annexe B** (schéma complet de **mini-pagila** avec ses contraintes nommées, et contenu des tables). Les tables sont inchangées depuis le TD1 ; seules les contraintes ont été explicitées et nommées. Sauf mention contraire, **chaque question repart de l'état initial** de l'annexe B.

Les six pièges de la séance

- P1.** Une contrainte CHECK n'est violée que si elle s'évalue à *faux*. Si elle vaut *inconnu*, la ligne passe : CHECK (`length > 0`) accepte une durée NULL.
- P2.** Un CHECK ne peut porter que sur la **ligne courante** : ni sous-requête, ni colonne d'une autre table. Une règle qui compare plusieurs lignes ou plusieurs tables n'est pas exprimable ainsi.
- P3.** Les modes ON DELETE / ON UPDATE se déclarent **clé étrangère par clé étrangère**. Deux tables filles de la même table mère peuvent avoir des comportements différents – et une seule suffit à faire échouer l'opération.
- P4.** Dans un déclencheur BEFORE ... FOR EACH ROW, la valeur renvoyée **est** la ligne qui sera écrite : RETURN NULL annule l'opération *sans aucun message*, et modifier NEW modifie la ligne enregistrée.
- P5.** Dans un déclencheur AFTER, la valeur renvoyée est **ignorée** : modifier NEW n'a aucun effet, et seule une exception peut encore annuler l'opération.
- P6.** NEW n'existe pas pour un DELETE et OLD n'existe pas pour un INSERT : la variable vaut alors NULL sans erreur. Conséquence : un RETURN NEW dans un BEFORE DELETE annule *toutes* les suppressions, silencieusement.

1 Contraintes natives

Exercice 1.1 – Lire et comprendre un schéma constraint

Voici le script de création de table `rental` de la base de données `mini-pagila` :

```
CREATE TABLE rental (
  rental_id    INTEGER PRIMARY KEY,
  rental_date  DATE NOT NULL,
  return_date  DATE,
  film_id      INTEGER NOT NULL CONSTRAINT fk_rental_film
              REFERENCES film(film_id) ON DELETE RESTRICT,
  customer_id  INTEGER NOT NULL CONSTRAINT fk_rental_customer
              REFERENCES customer(customer_id)
              ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT ck_rental_dates CHECK (return_date IS NULL OR return_date >= rental_date)
);
```

Pour chacune des instructions ci-dessous, dites si elle est acceptée ou rejetée, et comment les données évoluent. Si elle est rejetée, nommez la contrainte responsable.

```
(1) DELETE FROM film WHERE film_id = 1;
(2) INSERT INTO rental VALUES (19, '2025-04-01', '2025-03-01', 3, 1);
(3) INSERT INTO rental VALUES (9, '2025-02-01', '2025-03-01', 3, 1);
(4) UPDATE customer SET customer_id = 9 WHERE customer_id = 1;
```

Exercice 1.2 – Compléter le schéma

On souhaite ajouter à `mini-pagila` une table `review` enregistrant les avis des clients sur les films, avec les règles suivantes : un avis porte une note entière de 0 à 5 ; un client ne peut donner qu'un seul avis par film ; supprimer un film ou un client supprime ses avis ; la date de l'avis vaut par défaut la date du jour. Complétez :

```
CREATE TABLE review (
  review_id    INTEGER .....(1).....,
  film_id      INTEGER NOT NULL .....(2)..... film(film_id) .....(3).....,
  customer_id  INTEGER NOT NULL REFERENCES customer(customer_id) ON DELETE CASCADE,
  note         INTEGER NOT NULL CHECK (note .....(4)..... 0 AND 5),
  commentaire  TEXT,
  review_date  DATE NOT NULL .....(5)..... CURRENT_DATE,
  CONSTRAINT uq_review .....(6)..... (film_id, customer_id)
);
```

- Complétez les six trous.
- La contrainte `uq_review` empêche-t-elle un client de donner deux avis sur deux films différents ? Et deux clients de donner chacun un avis sur le même film ?
- On insère successivement les deux lignes suivantes. Que se passe-t-il ?

```
INSERT INTO review VALUES (1, 1, 1, 4, 'bien', '2025-04-01');
INSERT INTO review VALUES (2, 1, 1, 5, 'encore mieux', '2025-04-02');
```

Exercice 1.3 – Corriger le schéma

Pour chaque proposition : est-elle acceptée par le SGBD ? Si oui, produit-elle l'effet attendu ? Indiquez à quel moment le problème apparaît, et proposez une solution.

- Intention : garantir que toute location référence un client existant.*

```
ALTER TABLE rental ADD CONSTRAINT ck_rental_client
  CHECK (customer_id IN (SELECT customer_id FROM customer));
```

- b) *Intention : en cas de suppression d'un client, conserver ses locations en les « détachant ».*

```
ALTER TABLE rental DROP CONSTRAINT fk_rental_customer;  
ALTER TABLE rental ADD CONSTRAINT fk_rental_customer FOREIGN KEY (customer_id)  
REFERENCES customer(customer_id) ON DELETE SET NULL;
```

- c) *Intention : garantir que la durée d'un film est renseignée et strictement positive.*

```
ALTER TABLE film ADD CONSTRAINT ck_film_duree CHECK (length > 0);
```

Exercice 1.4 – Écrire les contraintes

Voici huit règles métier souhaitées par le gérant du vidéoclub. Pour chacune : **(i)** dites si elle est exprimable par une contrainte native ; **(ii)** si oui, écrivez l'instruction `ALTER TABLE` correspondante ; **(iii)** si non, expliquez précisément ce qui l'en empêche.

- a) Le tarif de location est strictement positif et ne dépasse pas 9,99 €.
- b) Un client ne peut pas avoir deux fois le même couple (prénom, nom).
- c) Supprimer un acteur supprime ses participations aux films.
- d) Un paiement ne peut pas dépasser trois fois le tarif du film loué.
- e) Un client ne peut pas avoir plus de trois locations en cours simultanément.
- f) Un film déjà loué et non rendu ne peut pas être loué à nouveau.

2 Déclencheurs

Anatomie d'un déclencheur

Un déclencheur se déclare en **deux temps** : une fonction sans paramètre qui renvoie le type spécial TRIGGER, puis le déclencheur qui l'associe à une table et à des événements.

```
CREATE OR REPLACE FUNCTION trg_ma_regle() RETURNS TRIGGER AS $$
BEGIN
    ...
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_ma_regle
BEFORE INSERT OR UPDATE ON rental      -- quand, et sur quelle table
FOR EACH ROW                          -- une exécution par ligne modifiée
EXECUTE FUNCTION trg_ma_regle();
```

Opération	OLD	NEW	Valeur à renvoyer (BEFORE ... FOR EACH ROW)
INSERT	NULL	la ligne à insérer	NEW (éventuellement modifiée)
UPDATE	l'ancienne	la nouvelle	NEW (éventuellement modifiée)
DELETE	la ligne visée	NULL	OLD

Renvoyer NULL depuis un déclencheur BEFORE ... FOR EACH ROW **annule l'opération pour cette ligne**, sans message ni erreur. Dans un déclencheur AFTER, la valeur renvoyée est ignorée (on écrit RETURN NULL par convention). TG_OP contient 'INSERT', 'UPDATE' ou 'DELETE'.

Exercice 2.1 – Lire un déclencheur (PL/pgSQL → français)

Pour chaque déclencheur : (i) énoncez en français ce qu'il garantit ou automatise ; (ii) répondez aux questions posées.

a)

```
CREATE OR REPLACE FUNCTION trg_payment_default() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.amount IS NULL THEN
        SELECT f.rental_rate INTO NEW.amount
        FROM rental r JOIN film f ON r.film_id = f.film_id
        WHERE r.rental_id = NEW.rental_id;
    END IF;
    IF NEW.payment_date IS NULL THEN
        NEW.payment_date := CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_payment_default BEFORE INSERT ON payment
FOR EACH ROW EXECUTE FUNCTION trg_payment_default();
```

Question : on insère un paiement pour la location 4, sans montant ni date. Quelle ligne est effectivement enregistrée ? Le déclencheur fonctionnerait-il en AFTER ?

b)

```
CREATE OR REPLACE FUNCTION trg_film_disponible() RETURNS TRIGGER AS $$
DECLARE v_nb INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_nb FROM rental
```

```

WHERE film_id = NEW.film_id AND return_date IS NULL AND rental_id <> NEW.
rental_id;
IF v_nb > 0 THEN
    RAISE EXCEPTION 'Le film % est déjà loué et non rendu', NEW.film_id;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_film_dispo BEFORE INSERT OR UPDATE ON rental
FOR EACH ROW EXECUTE FUNCTION trg_film_disponible();

```

Questions : quel est le rôle de la condition `rental_id <> NEW.rental_id`? Que donnent les deux insertions suivantes?

```

INSERT INTO rental VALUES (19, '2025-04-01', NULL, 3, 1);
INSERT INTO rental VALUES (20, '2025-04-01', NULL, 7, 1);

```

- c) Pour cette question, on ajoute à `customer` une colonne dénormalisée nommée `nb_en_cours` (entier, NOT NULL, défaut 0) destinée à contenir le nombre de locations non rendues du client. Elle est initialisée conformément aux données de l'annexe B :

	Dupont	Martin	Bernard	Petit	Moreau	Girard	Roche	Faure
nb_en_cours	0	2	0	1	0	1	0	0

```

CREATE OR REPLACE FUNCTION trg_maj_compteur() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.return_date IS NULL THEN
            UPDATE customer SET nb_en_cours = nb_en_cours + 1 WHERE customer_id = NEW.
customer_id;
        END IF;
    ELSIF TG_OP = 'DELETE' THEN
        IF OLD.return_date IS NULL THEN
            UPDATE customer SET nb_en_cours = nb_en_cours - 1 WHERE customer_id = OLD.
customer_id;
        END IF;
    ELSIF TG_OP = 'UPDATE' THEN
        IF OLD.return_date IS NULL AND NEW.return_date IS NOT NULL THEN
            UPDATE customer SET nb_en_cours = nb_en_cours - 1 WHERE customer_id = OLD.
customer_id;
        ELSIF OLD.return_date IS NOT NULL AND NEW.return_date IS NULL THEN
            UPDATE customer SET nb_en_cours = nb_en_cours + 1 WHERE customer_id = NEW.
customer_id;
        END IF;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_compteur AFTER INSERT OR UPDATE OR DELETE ON rental
FOR EACH ROW EXECUTE FUNCTION trg_maj_compteur();

```

Question : on exécute successivement

```

INSERT INTO rental VALUES (21, '2025-04-01', NULL, 7, 1);
UPDATE rental SET return_date = '2025-04-05' WHERE rental_id = 4;
DELETE FROM rental WHERE rental_id = 16;

```

Quelles sont les valeurs finales de `nb_en_cours` pour Dupont (1) et Martin (2)? Pourquoi RETURN NULL ne pose-t-il ici aucun problème?

Exercice 2.2 – Ce résultat est-il cohérent ?

1. On définit la fonction suivante, puis on l'associe à **payment** de trois façons différentes. Chaque cas repart de l'état initial.

```
CREATE OR REPLACE FUNCTION trg_filtre() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.amount = 0 THEN
        RETURN NULL;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Cas A

```
CREATE TRIGGER t_filtre BEFORE INSERT ON payment
FOR EACH ROW EXECUTE FUNCTION trg_filtre();

INSERT INTO payment VALUES (21, 1, NULL, 0, '2025-04-01');
```

État annoncé

L'insertion échoue avec l'erreur *null value returned by trigger*. La table **payment** contient toujours 14 lignes.

Cas B

```
CREATE TRIGGER t_filtre_after AFTER INSERT ON payment
FOR EACH ROW EXECUTE FUNCTION trg_filtre();

INSERT INTO payment VALUES (22, 1, NULL, 0, '2025-04-01');
```

État annoncé

Le paiement 22 est bien inséré : **payment** contient 15 lignes.

2. On utilise cette fois la fonction suivante :

```
CREATE OR REPLACE FUNCTION trg_arrondi() RETURNS TRIGGER AS $$
BEGIN
    NEW.amount := ROUND(NEW.amount);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

associée aux deux types de trigger BEFORE et AFTER comme précédemment, et l'on insère à chaque fois un paiement de 4,99 € :

```
CREATE TRIGGER t_arrondi_before BEFORE INSERT ON payment FOR EACH ROW EXECUTE FUNCTION
trg_arrondi();
CREATE TRIGGER t_arrondi_after AFTER INSERT ON payment FOR EACH ROW EXECUTE FUNCTION
trg_arrondi();
```

État annoncé

Dans les deux cas, le montant enregistré est 5.00 : le déclencheur arrondit la valeur avant écriture.

Exercice 2.3 – Compléter le déclencheur

Objectif : interdire qu'une date de retour soit postérieure à la date du jour (cf. exercice 1.4), à l'insertion comme à la modification.

```
CREATE OR REPLACE FUNCTION trg_retour_valide() .....(1)..... AS $$
BEGIN
  IF .....(2)..... IS NOT NULL AND .....(3)..... > CURRENT_DATE THEN
    RAISE .....(4)..... 'Date de retour dans le futur : %', NEW.return_date;
  END IF;
  RETURN .....(5).....;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_retour
.....(6)..... INSERT OR UPDATE ON rental
FOR EACH .....(7).....
EXECUTE FUNCTION trg_retour_valide();
```

- Complétez les sept trous.
- Que produit `UPDATE rental SET return_date = '2030-01-01' WHERE rental_id = 4;` ?
- Pourquoi ce déclencheur doit-il aussi couvrir `UPDATE`, et pas seulement `INSERT` ?
- * Aurait-on pu écrire cette règle avec un `CHECK` ? Discutez.

Exercice 2.4 – Corriger le déclencheur

Pour chaque déclencheur : le problème apparaît-il à la création ou à l'exécution ? Est-il signalé, ou silencieux ? Corrigez.

a)

```
CREATE OR REPLACE FUNCTION trg_journal() RETURNS INTEGER AS $$
BEGIN
  RAISE NOTICE 'nouvelle ligne';
  RETURN 1;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_journal BEFORE INSERT ON payment
FOR EACH ROW EXECUTE FUNCTION trg_journal();
```

- b) *Intention : empêcher la suppression d'une location non rendue.*

```
CREATE OR REPLACE FUNCTION trg_protege_location() RETURNS TRIGGER AS $$
BEGIN
  IF OLD.return_date IS NULL THEN
    RAISE EXCEPTION 'Impossible de supprimer une location en cours';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_protege BEFORE DELETE ON rental
FOR EACH ROW EXECUTE FUNCTION trg_protege_location();
```

- c) *Intention : journaliser dans la table `archive(customer_id, total_amount)` le montant total actuellement payé par chaque client.*

```
CREATE OR REPLACE FUNCTION trg_montant_total() RETURNS TRIGGER AS $$
BEGIN
  IF NOT EXISTS( SELECT * FROM archive WHERE customer_id = NEW.customer_id ) THEN
```

```
        INSERT INTO archive VALUES( NEW.customer_id, NEW.amount ) ;
    ELSE
        UPDATE archive
        SET total_amount = total_amount + NEW.amount
        WHERE customer_id = NEW.customer_id ;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_montant_total AFTER INSERT ON payment
FOR EACH ROW EXECUTE FUNCTION trg_montant_total();
```

Exercice 2.5 – Écrire le déclencheur

Implémentez les règles "métiers" jugées non exprimables nativement en 1.5. Pour chacune, précisez et justifiez BEFORE/AFTER, les événements couverts, et la valeur renvoyée.

- a) *Règle 8* : un film déjà loué et non rendu ne peut pas être loué à nouveau.
- b) *Règle 7* : un client ne peut pas avoir plus de trois locations en cours.
- c) * *Règle 6* : un paiement ne peut pas dépasser trois fois le tarif du film loué.
- d) * Le déclencheur `t_compteur` de l'exercice 2.1.c (destiné à maintenir le nombre de locations non rendues du client dans une colonne `nb_en_cours`) ne gère pas le changement de client d'une location (`UPDATE rental SET customer_id = ...`). Complétez-le.

A Aide-mémoire

1. Où déclarer une contrainte

```
-- à la création de la table, au niveau colonne
CREATE TABLE t (
  a INTEGER PRIMARY KEY,
  b INTEGER NOT NULL DEFAULT 0,
  c INTEGER UNIQUE,
  d INTEGER CONSTRAINT ck_d CHECK (d > 0),
  e INTEGER REFERENCES autre(cle) ON DELETE CASCADE
);
-- à la création de la table, au niveau table (obligatoire si plusieurs colonnes)
CREATE TABLE u (
  a INTEGER, b INTEGER,
  CONSTRAINT pk_u PRIMARY KEY (a, b),
  CONSTRAINT uq_u UNIQUE (a),
  CONSTRAINT fk_u FOREIGN KEY (a) REFERENCES autre(cle) ON UPDATE CASCADE
);
-- après coup
ALTER TABLE t ADD CONSTRAINT ck_t CHECK (b BETWEEN 0 AND 100);
ALTER TABLE t DROP CONSTRAINT ck_t;
ALTER TABLE t ALTER COLUMN b SET NOT NULL;      -- NOT NULL n'est pas une contrainte nommée
```

Nommer ses contraintes n'est pas obligatoire, mais les messages d'erreur deviennent lisibles et la suppression possible. Une contrainte ajoutée par `ALTER TABLE` est **vérifiée sur les données déjà présentes**.

2. Les cinq contraintes natives

NOT NULL	la colonne doit être renseignée.
PRIMARY KEY	identifie la ligne; équivaut à UNIQUE + NOT NULL. Une seule par table.
UNIQUE	pas deux fois la même valeur... mais plusieurs NULL sont autorisés (deux NULL ne sont pas « égaux »).
CHECK	condition sur la ligne courante uniquement. Violée seulement si elle vaut <i>faux</i> ; si elle vaut <i>inconnu</i> , la ligne passe. Ni sous-requête, ni autre table.
FOREIGN KEY	la valeur doit exister dans la table référencée (ou être NULL).

3. Modes d'intégrité référentielle

Se déclarent sur *chaque* clé étrangère, séparément pour `ON DELETE` (suppression de la ligne mère) et `ON UPDATE` (modification de sa clé).

NO ACTION	défaut : l'opération est rejetée s'il reste des lignes filles (contrôle en fin d'instruction).
RESTRICT	idem, mais le contrôle est immédiat et ne peut pas être différé.
CASCADE	les lignes filles sont supprimées (ou leur clé mise à jour) en même temps.
SET NULL	la clé étrangère des lignes filles est mise à NULL – <i>impossible si la colonne est NOT NULL</i> .
SET DEFAULT	la clé étrangère prend sa valeur par défaut, qui doit exister dans la table mère.

4. Déclencheurs

```
CREATE OR REPLACE FUNCTION trg_xxx() RETURNS TRIGGER AS $$
DECLARE ...
BEGIN
  ...
  RETURN NEW;
-- NEW, OLD, TG_OP sont disponibles ici
-- ou OLD, ou NULL
```

```

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER t_xxx
  { BEFORE | AFTER } { INSERT | UPDATE | DELETE } [ OR ... ] ON ma_table
  [ FOR EACH ROW | FOR EACH STATEMENT ]           -- défaut : FOR EACH STATEMENT
  [ WHEN (condition) ]                             -- filtre optionnel
  EXECUTE FUNCTION trg_xxx();

DROP TRIGGER t_xxx ON ma_table;
ALTER TABLE ma_table DISABLE TRIGGER t_xxx;

```

BEFORE ROW	s'exécute avant l'écriture. La valeur renvoyée est la ligne écrite : on peut la modifier (NEW.x := ...) ou annuler l'opération (RETURN NULL). Renvoyer NEW en INSERT/UPDATE, OLD en DELETE.
AFTER ROW	s'exécute après l'écriture. Valeur de retour ignorée (RETURN NULL par convention); modifier NEW est sans effet. Seule une exception peut encore tout annuler. Usage typique : maintenir une donnée dérivée, journaliser.
STATEMENT	une seule exécution par instruction; NEW et OLD n'y sont pas définis.

RAISE EXCEPTION 'message %', var; interrompt l'opération *et* annule la transaction en cours. Toute la syntaxe PL/pgSQL du TD2 (SELECT ... INTO, IF, boucles, blocs EXCEPTION) reste utilisable dans une fonction déclencheur.

B La base mini-pagila et ses contraintes

Les tables et les données sont **identiques** à celles des **TD1** et **TD2**; seules les contraintes ont été explicitées et nommées.

Schéma complet

```
CREATE TABLE category (
  category_id INTEGER PRIMARY KEY,
  name        VARCHAR(25) NOT NULL CONSTRAINT uq_category_name UNIQUE
);

CREATE TABLE actor (
  actor_id   INTEGER PRIMARY KEY,
  first_name VARCHAR(45) NOT NULL,
  last_name  VARCHAR(45) NOT NULL
);

CREATE TABLE customer (
  customer_id INTEGER PRIMARY KEY,
  first_name  VARCHAR(45) NOT NULL,
  last_name   VARCHAR(45) NOT NULL,
  city        VARCHAR(45),
  active      BOOLEAN NOT NULL DEFAULT TRUE,
  create_date DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE film (
  film_id      INTEGER PRIMARY KEY,
  title         VARCHAR(60) NOT NULL,
  release_year  INTEGER,
  length        INTEGER CONSTRAINT ck_film_length CHECK (length > 0),
  rating        VARCHAR(6) CONSTRAINT ck_film_rating
               CHECK (rating IN ('G','PG','PG-13','R','NC-17')),
  rental_rate   NUMERIC(4,2) NOT NULL CONSTRAINT ck_film_rate CHECK (rental_rate > 0),
  category_id   INTEGER CONSTRAINT fk_film_category REFERENCES category(category_id)
               ON DELETE SET NULL ON UPDATE CASCADE
);

CREATE TABLE film_actor (
  film_id  INTEGER CONSTRAINT fk_fa_film REFERENCES film(film_id) ON DELETE CASCADE,
  actor_id INTEGER CONSTRAINT fk_fa_actor REFERENCES actor(actor_id) ON DELETE CASCADE,
  CONSTRAINT pk_film_actor PRIMARY KEY (film_id, actor_id)
);

CREATE TABLE rental (
  rental_id   INTEGER PRIMARY KEY,
  rental_date DATE NOT NULL,
  return_date DATE,
  film_id     INTEGER NOT NULL CONSTRAINT fk_rental_film
               REFERENCES film(film_id) ON DELETE RESTRICT,
  customer_id INTEGER NOT NULL CONSTRAINT fk_rental_customer
               REFERENCES customer(customer_id)
               ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT ck_rental_dates CHECK (return_date IS NULL OR return_date >= rental_date)
);

CREATE TABLE payment (
  payment_id INTEGER PRIMARY KEY,
```

```
customer_id  INTEGER NOT NULL CONSTRAINT fk_pay_customer
              REFERENCES customer(customer_id) ON DELETE CASCADE,
rental_id    INTEGER CONSTRAINT fk_pay_rental
              REFERENCES rental(rental_id) ON DELETE SET NULL,
amount       NUMERIC(5,2) NOT NULL CONSTRAINT ck_pay_amount CHECK (amount >= 0),
payment_date DATE NOT NULL
);
```